

8 Capabilities

8.1 Evaluation summary

Evaluation		Claude models		Other models	
		Claude Sonnet 5	Claude Sonnet 4.6	GPT-5.5	Gemini 3.5 Flash
SWE-bench Pro		63.2	58.1	58.6	55.1
Terminal-Bench 2.1		80.4	67.0	83.4 (Codex CLI)	76.2
BrowseComp		84.7 (single agent) 86.6 (multi agent)	76.2	84.4	-
Humanity's Last Exam	No tools	43.2	34.6	41.4	40.2
	With tools	57.4	46.8	52.2	-
OSWorld-Verified ¹⁴		81.2	78.5	78.7	78.4
FrontierCode v1		38.8	15.1	25.5	-
GDPval-AA v2 ¹⁵		1618	1395	1509	1357
AutomationBench		13.5	5.3	12.9	14.5
Legal Agent Benchmark	Full Public Set	8.9	8.0	-	-
	Harvey's Held-Out Set	5.8	5.4	2.1	0.8
HealthBench Professional		57.8	44.2	51.8	-

¹⁴ Changes to the Sonnet OSWorld score are due to a bug fix on our zoom tool when paired with batched actions, and increasing the max tokens per turn from 16K to 128K.

¹⁵ Elo score as of June 17, 2026.

[Table 8.1.A] Capability evaluation summary. Unless otherwise noted, all Claude Sonnet 5 results use the following standard configuration: adaptive thinking at max effort, default sampling settings (temperature, top_p), averaged over 5 trials. Context window sizes are evaluation-dependent. Standard configurations use 1M tokens; BrowseComp uses a 10M-token limit with context compaction (triggered at 200k). Other evaluations range from 300k to 1M tokens. The best score in each row is **bolded**. Competitor figures are drawn from the respective developers' published system cards or benchmark leaderboards.

8.2 SWE-bench Verified, Pro, Multilingual, and Multimodal

SWE-bench (Software Engineering Bench) tests AI models on real-world software engineering tasks. We report four variants, where the score is the average over five trials:

- SWE-bench **Verified**¹⁶ is a 500-problem subset, each verified by human engineers as solvable. Claude Sonnet 5 achieved 85.2%.
- SWE-bench **Pro**¹⁷ is a harder variant: problems drawn from actively-maintained repositories with larger, multi-file diffs and reduced public ground-truth leakage. Sonnet 5 achieved 63.2%.
- SWE-bench **Multilingual** extends the format to 300 problems across 9 programming languages. Sonnet 5 achieved 78.3%.
- SWE-bench **Multimodal**¹⁸ adds visual context (screenshots, design mockups) to the issue descriptions (see Section 9.3 of the [Claude Opus 4.7 System Card](#) for details on the internal harness). Sonnet 5 achieved 28.1%.

All SWE-bench variants use the standard configuration, with thinking blocks included in the sampling results. For our memorization screening, see Section 6.2.1 in the [Mythos Preview System Card](#).

8.3 Terminal-Bench 2.1

Terminal-Bench 2.1¹⁹ tests AI models on real-world coding tasks in terminal and command-line environments. We're using [mini-SWE-agent](#) as a harness, as it's more robust to timeouts compared to the Terminus-2. At **xhigh** effort, Terminus-2 experiences 2.7× more timeouts than mini-SWE-agent, due to the way it waits for command execution through a tmux session; this makes final scores noisier and less legible.

¹⁶ Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770. <https://arxiv.org/abs/2310.06770>

¹⁷ Deng, X., et al. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941. <https://arxiv.org/abs/2509.16941>

¹⁸ Yang, J., et al. (2024). SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains? arXiv:2410.03859. <https://arxiv.org/abs/2410.03859>

¹⁹ Merrill, M. A., et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868. <https://arxiv.org/abs/2601.11868>

On a GKE cluster with 1× timeout rate and 3× memory ceiling before pod preemption, Claude Sonnet 5 achieves 80.4% mean reward, averaged over 5 attempts for each one of the 89 unique tasks (for a total of 445 trials), at **xhigh** effort.

Sonnet 4.6 achieved a 67% score on the same evaluation and same infrastructure, at **high** effort.

8.4 FrontierCode

FrontierCode²⁰ is an agentic coding benchmark of 150 software engineering tasks created by Cognition. Tasks are derived from real pull requests in open-source repositories: e.g. fixing websocket bugs in the Python framework **aiohttp**, hardening Prisma’s browser bundle, or extending JSON schema linting rules.

Each task gives the agent a checked-out repository and a single issue description; the agent then works autonomously in a containerized environment to produce a final patch, with no human intervention and no timeout information. Patches are graded against blocking functional criteria (primarily held-out unit tests) plus weighted rubric criteria, including model-graded checks for required test coverage and prohibited implementation patterns. Tasks were authored by maintainers of the underlying repositories and individually reviewed by Cognition researchers, with a random subset manually solved to verify fairness.

²⁰ Lu, E., et al. (2026). Introducing FrontierCode. Cognition. <https://cognition.ai/blog/frontier-code>